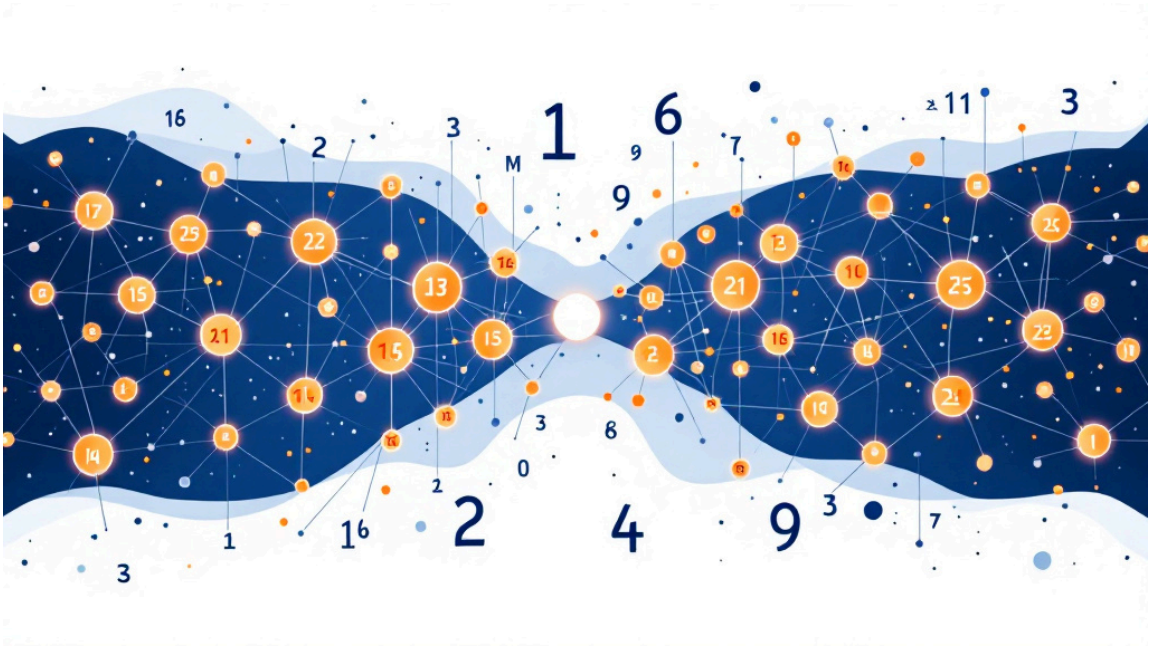
LABORATORIUM 5

Uczenie maszynowe i sztuczne sieci neuronowe

Przedmiot	Sztuczna inteligencja
Forma	Laboratorium
Czas	4 h dydaktyczne (190 min), 15.12.2026
Grupy	INLS5_SI
Prowadzący	mgr inż. Szymon Guzik

Cele zajęć

- Opisać neuron, funkcję aktywacji, warstwy i propagację wsteczną.
- Zaimplementować od zera sieć 2-4-1 i nauczyć ją funkcji XOR.
- Wytrenować perceptron wielowarstwowy w scikit-learn i ocenić go na zbiorze testowym.
- Rozpoznać przeuczenie i dobrać hiperparametry walidacją.
- Przygotować projekt do oddania (etap 4).



Sieć neuronowa — warstwy neuronów uczą się z przykładów

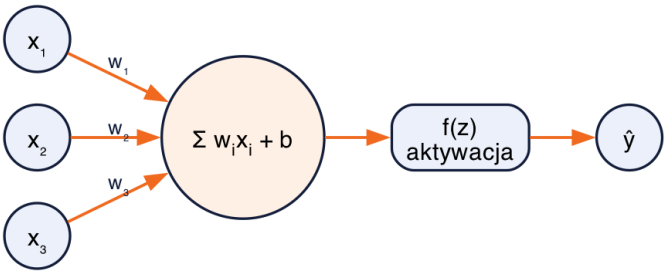
1. Plan zajęć (190 minut)

Czas	Co robimy
0–25 min	Uczenie maszynowe: nadzorowane, nienadzorowane; neuron i sieć
25–70 min	Sieć od zera w NumPy — XOR, propagacja wsteczna, zadania 1–2
70–80 min	Przerwa
80–130 min	scikit-learn: rozpoznawanie cyfr, ocena modelu, zadania 3–4
130–190 min	Projekt — etap 4: przygotowanie do oddania, próba prezentacji

2. Teoria

Uczenie maszynowe

Zamiast pisać reguły ręcznie (jak w systemie ekspertowym), pokazujemy programowi **przykłady** i pozwalamy mu dobrać parametry modelu tak, żeby błąd na przykładach był jak najmniejszy. **Uczenie nadzorowane** — mamy wejścia i poprawne odpowiedzi (klasyfikacja, regresja). **Nienadzorowane** — same dane, szukamy struktury (grupowanie). **Ze wzmocnieniem** — agent dostaje nagrody za działania.

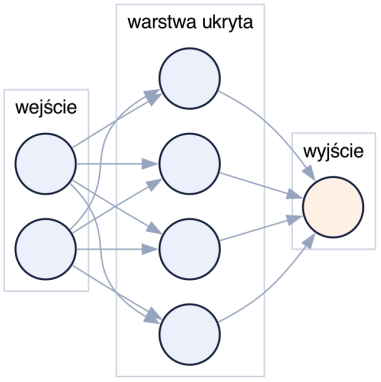


Pojedynczy neuron — suma ważona i funkcja aktywacji

Neuron liczy sumę ważoną wejść plus przesunięcie (bias) i przepuszcza ją przez **funkcję aktywacji**:

Funkcja	Wzór	Zastosowanie
Sigmoida	$1 / (1 + e^{-z})$	wyjście binarne, klasyczne sieci
tanh	$(e^{z} - e^{-z}) / (e^{z} + e^{-z})$	warstwy ukryte, wartości −1…1
ReLU	$\max(0, z)$	standard w głębokich sieciach
Softmax	$e^{z_i} / \Sigma e^{z_j}$	wyjście wieloklasowe — prawdopodobieństwa

Uczenie to minimalizacja funkcji straty metodą **spadku gradientu**. Gradient względem wag liczymy **propagacją wsteczną** (*backpropagation*): błąd z wyjścia przenosimy warstwa po warstwie do tyłu, korzystając z reguły łańcuchowej.



Architektura 2-4-1 z przykładu XOR

3. Przykład 1 — sieć od zera: XOR

Pojedynczy neuron nie nauczy się XOR (punkty nie są liniowo separowalne) — to słynna krytyka Minsky'ego i Paperta z 1969 r. Wystarczy jednak jedna warstwa ukryta.

mlp_xor.py

```python
"""Sieć neuronowa od zera (NumPy): 2–4–1, sigmoida, propagacja wsteczna – uczy się funkcji XOR."""
import numpy as np

rng = np.random.default_rng(0)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([[0], [1], [1], [0]], dtype=float)

W1, b1 = rng.normal(0, 1, (2, 4)), np.zeros((1, 4))
W2, b2 = rng.normal(0, 1, (4, 1)), np.zeros((1, 1))
sigmoid = lambda z: 1 / (1 + np.exp(-z))
lr = 1.0

for epoka in range(5001):
    h = sigmoid(X @ W1 + b1)                # propagacja w przód: warstwa ukryta
    out = sigmoid(h @ W2 + b2)              # wyjście
    blad = out - y
    d_out = blad * out * (1 - out)          # propagacja wsteczna (pochodna sigmoidy: s·(1−s))
    d_h = (d_out @ W2.T) * h * (1 - h)
    W2 -= lr * h.T @ d_out;  b2 -= lr * d_out.sum(0, keepdims=True)
    W1 -= lr * X.T @ d_h;    b1 -= lr * d_h.sum(0, keepdims=True)
    if epoka % 1000 == 0:
        print(f"epoka {epoka:4}: błąd MSE = {np.mean(blad ** 2):.4f}")

print("wyjścia sieci:", np.round(out.ravel(), 3), "-> klasy:", (out.ravel() > 0.5).astype(int))

```

kod/mlp_xor.py

```
$ python3 -B kod/mlp_xor.py
epoka    0: błąd MSE = 0.3430
epoka 1000: błąd MSE = 0.0033
epoka 2000: błąd MSE = 0.0011
epoka 3000: błąd MSE = 0.0006
epoka 4000: błąd MSE = 0.0004
epoka 5000: błąd MSE = 0.0003
wyjścia sieci: [0.007 0.981 0.981 0.024] -> klasy: [0 1 1 0]
```

Zadanie 1 (15 min)

Zmień współczynnik uczenia `lr` na 0,1 i 5,0 oraz liczbę neuronów ukrytych na 2 i 8. Zapisz w tabeli błąd po 5000 epokach. Co się dzieje przy `lr = 5,0`?

Zadanie 2 (20 min)

Zamień sigmoidę w warstwie ukrytej na **tanh** (pochodna: 1 – tanh²). Czy sieć uczy się szybciej? Narysuj krzywą błędu dla obu wariantów na jednym wykresie.

4. Przykład 2 — rozpoznawanie cyfr w scikit-learn

Zbiór `digits` zawiera 1797 obrazków 8×8 pikseli. Dzielimy go na część uczącą (75%) i testową (25%) — **model oceniamy wyłącznie na danych, których nie widział w trakcie uczenia**.

cyfry_sklearn.py

```python
"""Rozpoznawanie odręcznych cyfr 8x8 perceptronem wielowarstwowym (scikit-learn) + macierz pomyłek."""
import sys

from sklearn.datasets import load_digits
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_digits(return_X_y=True)                          # 1797 obrazków 8x8 = 64 cechy
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)

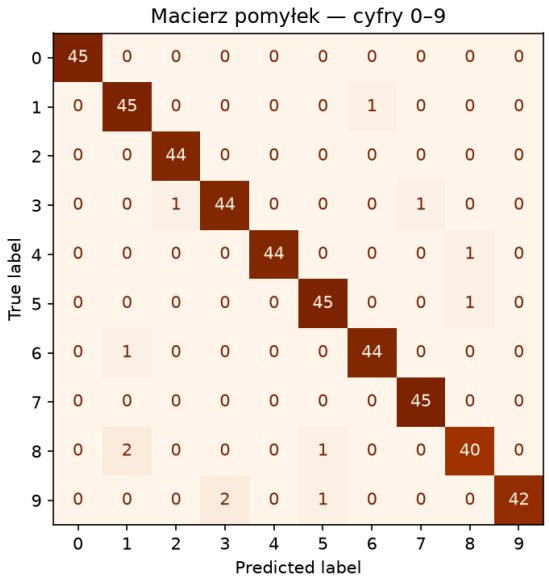
model = make_pipeline(StandardScaler(),
                      MLPClassifier(hidden_layer_sizes=(64,), max_iter=500, random_state=0))
model.fit(X_tr, y_tr)
pred = model.predict(X_te)
print(f"zbiór uczący: {len(X_tr)}, testowy: {len(X_te)}")
print(f"dokładność na zbiorze testowym: {accuracy_score(y_te, pred):.3f}")

if len(sys.argv) > 1:
    import matplotlib.pyplot as plt
    disp = ConfusionMatrixDisplay.from_predictions(y_te, pred, cmap="Oranges", colorbar=False)
    disp.ax_.set_title("Macierz pomyłek — cyfry 0–9")
    disp.figure_.set_size_inches(5.5, 5)
    disp.figure_.tight_layout()
    disp.figure_.savefig(sys.argv[1], dpi=150)

```

kod/cyfry_sklearn.py

```
$ python3 -B kod/cyfry_sklearn.py macierz_pomylek.png
zbiór uczący: 1347, testowy: 450
dokładność na zbiorze testowym: 0.973
```



Macierz pomyłek na zbiorze testowym (wygenerowana przez program z przykładu 2)

Zadanie 3 (20 min)

Porównaj trzy architektury: `(16,)`, `(64,)`, `(128, 64)`. Użyj `cross_val_score` (5-krotna walidacja krzyżowa) i wypisz średnią dokładność ± odchylenie. Którą wybierasz i dlaczego nie największą?

Zadanie 4 (15 min)

Znajdź w macierzy pomyłek najczęściej mylone cyfry i wyświetl 5 błędnie sklasyfikowanych obrazków (`plt.imshow(X_te[i].reshape(8, 8), cmap="gray")`). Czy człowiek też by się pomylił?

Przeuczenie

Model, który ma bardzo mały błąd na danych uczących, a duży na testowych, jest **przeuczony** — nauczył się przykładów na pamięć. Pomaga: więcej danych, mniejsza sieć, regularyzacja (`alpha` w `MLPClassifier`), wczesne zatrzymanie (`early_stopping=True`).

5. Projekt — etap 4

Przygotowanie do oddania (60 min)

Lista kontrolna: działa po `git clone` i instrukcji z README · min. dwie metody SI · testy przechodzą · raport 3–5 stron (problem, metoda, wyniki, wnioski) · prezentacja 10 min. **Termin oddania w Moodle: dzień przed ostatnimi zajęciami (11.01.2027, 23:59).** Dziś każdy zespół robi próbną prezentację w 5 minut.

6. Literatura

- G. Bonaccorso, *Algorytmy uczenia maszynowego*, Helion 2019.
- Dokumentacja scikit-learn: *Neural network models (supervised)*.
- S. Russell, P. Norvig, *AI: A Modern Approach* — rozdz. „Deep Learning”.